Estructuras de Datos

Parcial – Guardia con prioridades

La guardia de un hospital atiende a los pacientes según la gravedad de su caso. Cada paciente que llega recibe una **prioridad**, que es un número entre `0` y `P-1`, donde `0` es la más urgente. Siempre se atiende primero a alguien de la prioridad más urgente que tenga gente esperando, y entre pacientes de la misma prioridad se respeta el orden de llegada. Los pacientes se representan por su nombre, y no hay dos pacientes con el mismo nombre. El tipo abstracto `Guardia` tiene, entre otras, las siguientes operaciones (N es la cantidad de pacientes en espera):

- **`Guardia nuevaGuardia(int p)`**, que crea una guardia vacía con `p` niveles de prioridad. *Precondición:* `p > 0`. O(P).
- **`void llegar(string nombre, int prio, Guardia g)`**, que agrega al paciente al final de la espera de su prioridad. *Precondiciones:* `0 <= prio < P` y el paciente no está en espera. O(1).
- **`string atender(Guardia g)`**, que quita de la espera al paciente que corresponde atender y devuelve su nombre. *Precondición:* hay al menos un paciente en espera. O(P).
- **`void empeorar(string nombre, Guardia g)`**, que indica que el paciente empeoró: pasa a la prioridad inmediatamente más urgente, **al final** de la espera de esa prioridad. *Precondiciones:* el paciente está en espera y su prioridad es mayor que 0. O(N).
- **`int enEspera(Guardia g)`**, que indica cuántos pacientes hay esperando. O(1).

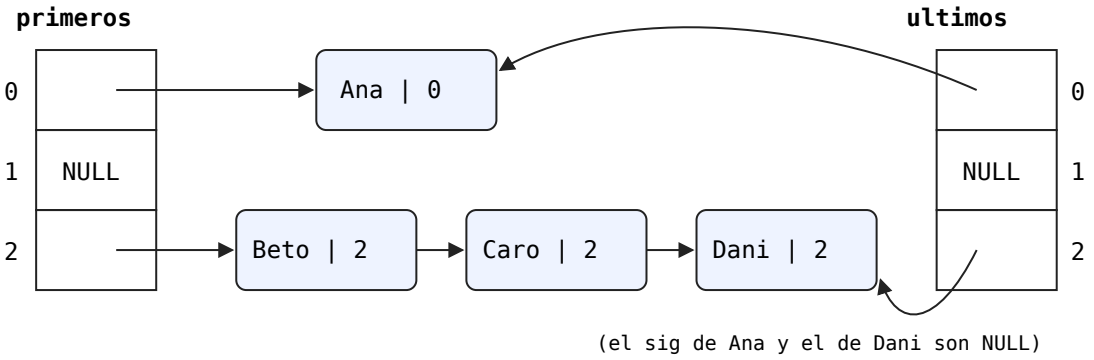
Para implementar el TAD se utilizará la siguiente representación:

```
struct PacienteNode {
    string nombre;
    int prio;               // prioridad actual del paciente
    PacienteNode* sig;      // siguiente en la espera de SU prioridad
};

struct GuardiaStr {
    int P;                  // cantidad de niveles de prioridad
    PacienteNode** primeros;  // array de P punteros: primero de cada prioridad
    PacienteNode** ultimos;   // array de P punteros: último de cada prioridad
    int cant;               // total de pacientes en espera
};

typedef GuardiaStr* Guardia;
```

Es decir, cada prioridad `i` tiene su propia cola, implementada como lista simplemente enlazada que empieza en `primeros[i]` y termina en `ultimos[i]`. Los arrays `primeros` y `ultimos` se piden con `new PacienteNode*[P]`. En el siguiente gráfico, una guardia con P = 3: Ana tiene prioridad 0, nadie tiene prioridad 1, y Beto, Caro y Dani tienen prioridad 2 (en ese orden de llegada).



Se proveen implementadas las siguientes funciones:

```
Guardia nuevaGuardia(int p) {
    Guardia g = new GuardiaStr;
    g->P = p;
    g->cant = 0;
    g->primeros = new PacienteNode*[p];
    g->ultimos  = new PacienteNode*[p];
    for (int i = 0; i < p; i++) {
        g->primeros[i] = NULL;
        g->ultimos[i]  = NULL;
    }
    return g;
}

int enEspera(Guardia g) {
    return g->cant;
}
```

Ejercicios

> **Importante:** en todos los ejercicios se evalúa el uso eficiente de recursos y la justificación de los costos. No se permite usar recursión.

1. Escribir los **invariantes de representación** de `GuardiaStr`.
2. Implementar, como implementador del TAD, las operaciones **`llegar`** y **`atender`**, con los costos indicados.
3. Implementar **`empeorar`** en O(N). Para desenganchar al paciente de su cola actual, **se debe recorrer la cola usando un puntero a puntero** (`PacienteNode** pp`), de forma que el caso en que el paciente es el primero de la cola **no** se trate aparte. Tener en cuenta qué pasa con `ultimos`.

   > **AYUDA:** si `pp` arranca valiendo `&g->primeros[i]`, entonces `*pp` es el primer nodo, y hacer `*pp = algo` cambia directamente `g->primeros[i]`. Para avanzar: `pp = &(*pp)->sig`.

4. Implementar como **usuario** del TAD la función **`int atenderHasta(string nombre, Guardia g)`**, que atiende pacientes hasta atender al de nombre dado (inclusive) y devuelve cuántos pacientes fueron atendidos. *Precondición:* el paciente está en espera. Indicar su costo en el peor caso.
5. Implementar **`void liberarGuardia(Guardia g)`**, que libera **toda** la memoria usada por la guardia, sin dejar memory leaks. Prestar atención a cómo se libera un array pedido con `new[]`.
6. Responder justificando: ¿se podría lograr que `atender` sea O(1) cambiando la representación? ¿Qué habría que agregar y cómo afectaría a las demás operaciones?